Spybee Frontend Technical Test

Candidate: Alejandro Andrade
Role: Frontend Developer - Next.js / React
Repository: https://github.com/AlejandroAndrade98/spybee-frontend-test
Live demo: https://spybee-frontend-test.vercel.app/login
Video walkthrough: https://youtu.be/kXNBrz8_f1E

---

1. Solution overview

This project implements a frontend application for construction incident management based on the Spybee technical test requirements.

The application includes an interactive incident map, an incident creation flow, a dashboard with operational summaries, advanced statistics, demo authentication, language/theme preferences and frontend-only attachment handling.

The implementation focuses on clean UI, maintainable structure, typed data, clear state management and a functional user experience.

---

2. Main features

- Demo authentication with protected routes.
- Interactive Mapbox map.
- Incident markers from the provided mock data.
- Incident creation flow based on map location selection.
- New incidents persisted locally.
- Incident creation form with category, priority, assignees, observers and attachments.
- Dashboard with KPIs, filters, summaries and paginated table.
- Advanced statistics view.
- Light/dark theme.
- Spanish/English UI.
- Options page for preferences.
- Remote-first data loading with local fallback.
- Docker support as optional runtime.

---

3. Technical stack

- Next.js App Router
- React
- TypeScript
- Zustand
- Mapbox GL
- SCSS Modules
- CSS variables
- Docker
- Vercel

---

4. Key technical decisions

Next.js App Router

The application is organized by routes:

- `/login`
- `/map`
- `/dashboard`
- `/options`

The home route redirects to `/login`, and protected pages are handled through an `AuthGuard`.

Zustand

Zustand is used because the app needs shared state between the map, modal, dashboard, header and options page.

Stores were separated by responsibility:

- `incidents.store`
- `auth.store`
- `preferences.store`

Mapbox GL

Mapbox objects such as the map instance, markers and popups are kept in `useRef`, not in Zustand. Zustand only stores application state such as incidents, selected incident and creation mode.

The mock data provides coordinates as:

```
{
  lat: number;
```

```
  lng: number;
}
```

Mapbox expects:

[lng, lat]

So the coordinates are converted before rendering markers.

Remote/local data strategy

The frontend does not consume the signed mock URL directly. Instead, it calls `/api/incidents`.

That route first tries `INCIDENTS_REMOTE_URL`. If the remote URL fails, expires or is missing, the app uses `public/data/incidents.mock.json`.

This keeps the prototype stable during review.

---

5. Architecture diagram

[Insert Architecture Diagram here]

The application is structured around protected routes, shared layout components and Zustand stores.

Main flow:

1. The user logs in through `/login`.
2. `AuthGuard` protects `/map`, `/dashboard` and `/options`.
3. UI components consume Zustand stores.
4. Incident data is loaded through `incidents.service.ts`.
5. `/api/incidents` resolves the remote data or local fallback.
6. Locally created incidents are persisted in the browser.

---

6. Incident creation flow

[Insert Incident Creation Flow Diagram here]

The creation flow follows the reference demo:

1. User clicks the global `+` button or `Create incident`.
2. The app navigates to `/map`.
3. Location picking mode starts.
4. User clicks on the map.
5. Coordinates are stored.
6. The creation modal opens.
7. User completes incident details.
8. Optional assignees, observers and attachments can be added.
9. The incident is saved in Zustand.
10. The new marker appears on the map.
11. The dashboard updates automatically.

---

7. Dashboard

The dashboard is divided into two modes:

Summary

Designed for fast operational review:

- Total visible incidents.
- Open, closed and paused incidents.
- High priority and overdue incidents.
- Incidents with evidence.
- Filters and paginated incident table.

Statistics

Designed for deeper analysis:

- Trends.
- Risk indicators.
- Distribution by category and tags.
- Team performance.

The table is paginated to avoid rendering the full mock dataset at once.

---

8. Demo authentication

Authentication is implemented as frontend-only demo functionality.

Demo users:

User	Password	Role
julian@spybee.com	demo123	Super admin
alejandro@spybee.com	demo123	Admin
carol@spybee.com	demo123	Inspector

Important notes:

- This is not production authentication.
- It is included as an extra feature for the technical test.
- Sessions persist locally.
- New incidents use the authenticated user as owner.
- Existing mock incidents keep their original owners, assignees and observers.

---

9. Attachments

The incident form includes frontend-only attachment handling.

Images can be previewed and stored locally as part of the created incident metadata. This allows the dashboard to count created incidents with evidence.

In a real product, attachments should be uploaded to a backend or storage service such as S3, Supabase Storage or another secure asset service.

---

10. Deployment and execution

The app is deployed on Vercel:

https://spybee-frontend-test.vercel.app/login

The repository includes Docker support as an optional way to run the production build locally.

Environment variables:

INCIDENTS_REMOTE_URL=
NEXT_PUBLIC_MAPBOX_TOKEN=

---

11. Future improvements

If this prototype became a production product, the next improvements would be:

- Real backend persistence.
- Production authentication.
- Role-based permissions.
- Server-side file storage.
- Incident editing.
- Status workflow.
- Marker clustering.
- Exportable reports.
- Audit trail.
- Automated tests.

---

11. Flowcharts

Diagram 1 — General Architecture

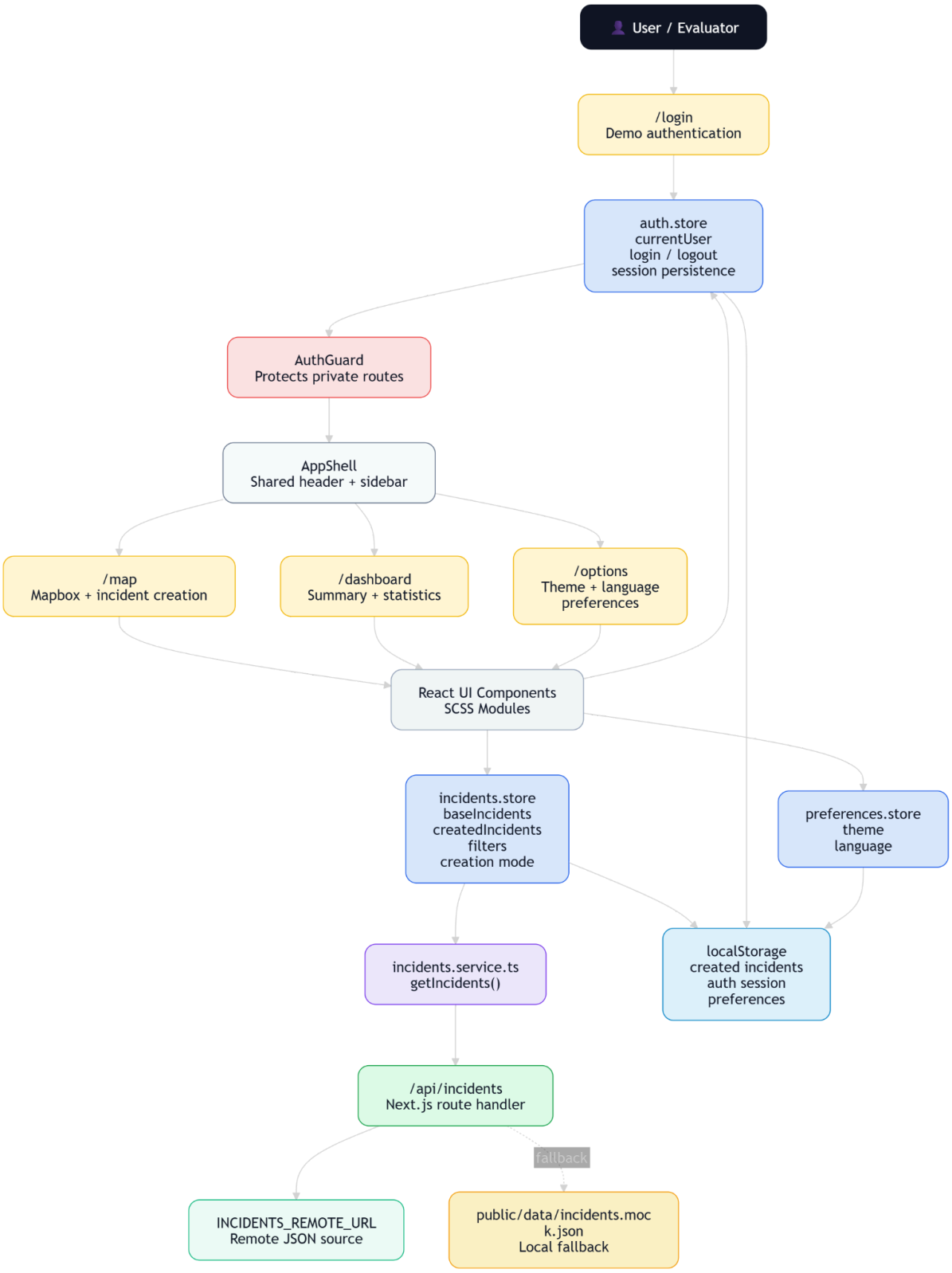

Diagram 2 — Incident Creation Flow

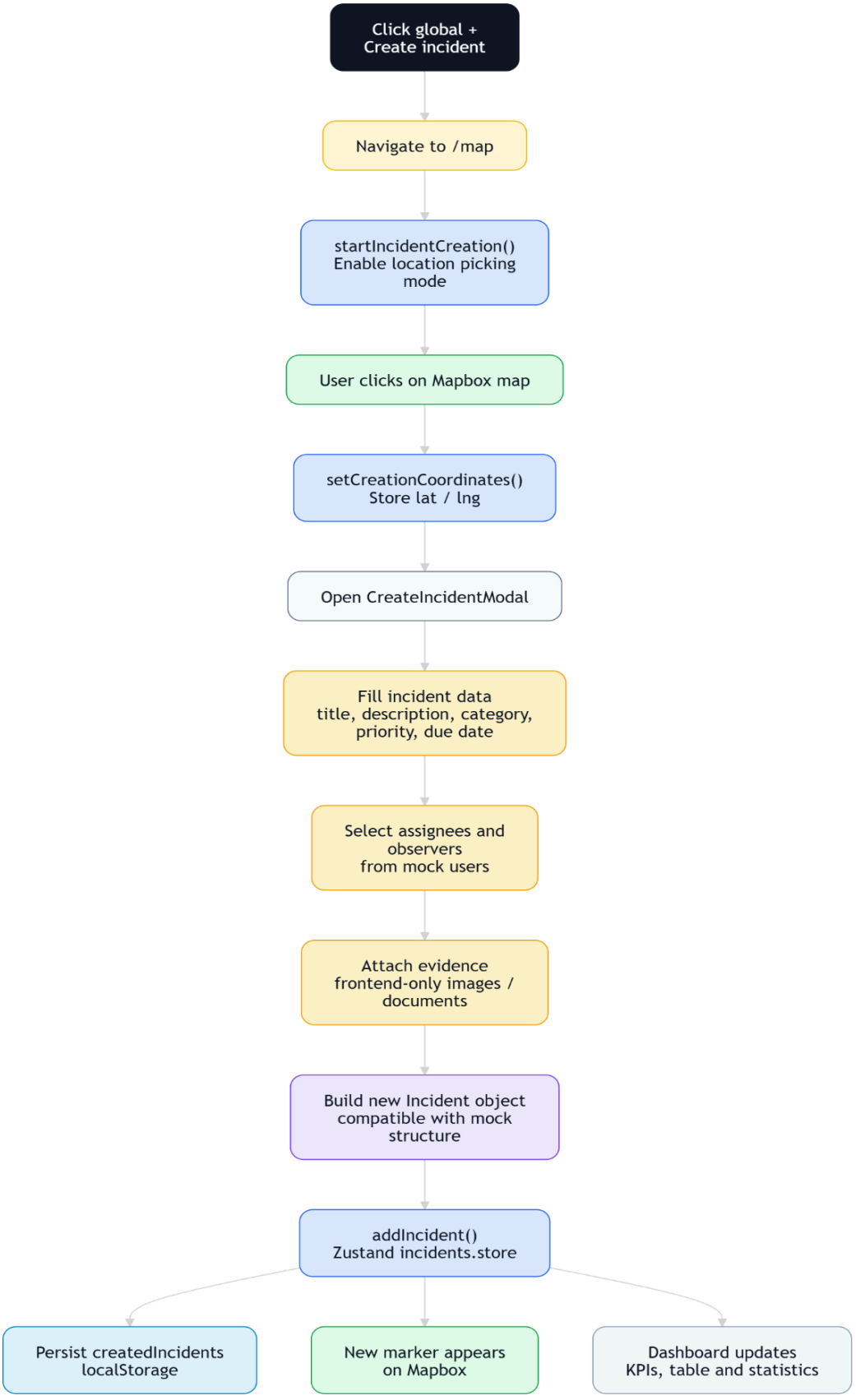

Diagram 3 — Remote/Local Data Strategy

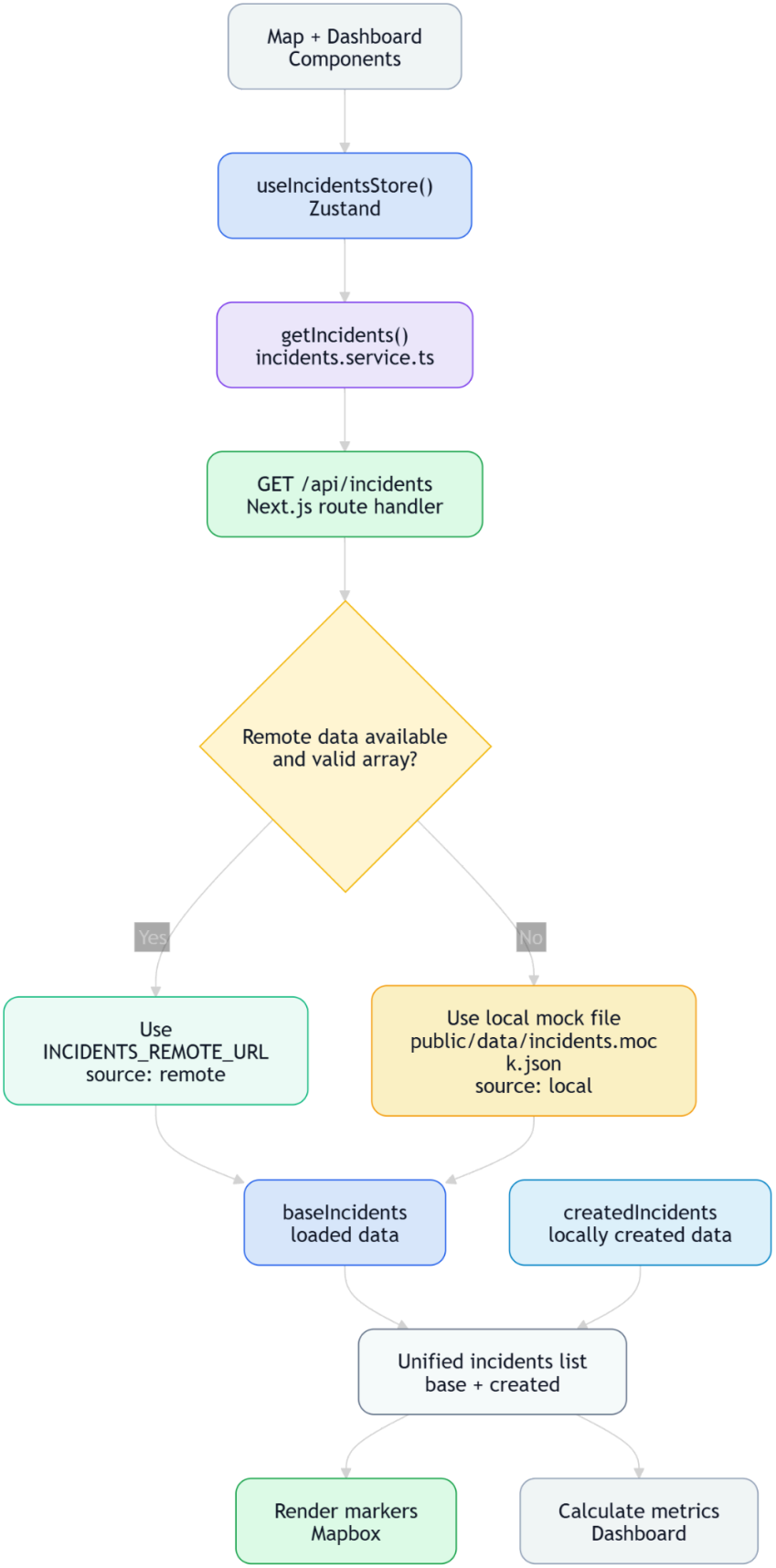

Diagram 4 — Demo Authentication and Protected Routes